

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital · Ingeniería de Software

Desarrollo de Aplicaciones Web con ASP.NET Core 10

Guía completa del estudiante — paso a paso con Visual Studio Code

Del primer `dotnet run` a una Web API CRUD segura con EF Core, PostgreSQL y JWT

Asignatura: Aplicaciones Web (5.º semestre) — Unidad II

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

IDE de referencia: Visual Studio Code + extensión C# Dev Kit

Versiones de referencia (.NET 10 LTS, C# 14, ASP.NET Core 10, EF Core 10, PostgreSQL 18): se detallan en la sección 2.

Contenido

1	Introducción	4
1.1	¿Qué es ASP.NET Core y por qué dominarlo?	4
1.2	C# para quien ya sabe Java	4
1.3	Cómo funciona ASP.NET Core en la web	4
1.4	Cómo usar esta guía y los niveles de los ejercicios	5
2	Preparación del entorno de desarrollo	5
2.1	Versiones de referencia de esta guía	5
2.2	Paso 1: instalar el SDK de .NET 10	6
2.3	Paso 2: instalar Visual Studio Code y las extensiones de C#	6
2.4	Paso 3: instalar PostgreSQL 18	7
2.5	Paso 4: comprobar todo el entorno con un proyecto de prueba	7
3	Fundamentos de C# para la web	8
3.1	Tipos, variables y propiedades	8
3.2	Métodos, lambdas y asincronía	9
3.3	Colecciones y LINQ	9
3.4	Ejercicios — fundamentos	9
4	Primer proyecto web: Hola Mundo con Minimal API	10
4.1	Crear y abrir el proyecto en VS Code	10
4.2	Anatomía del archivo de proyecto (.csproj)	10
4.3	El punto de entrada: Program.cs	11
4.4	Ejecutar y depurar desde VS Code	11
4.5	Ejercicios — primer proyecto	12
5	El modelo de hosting y el pipeline de middleware	12
5.1	Qué es un middleware y por qué importa el orden	12
5.2	Inyección de dependencias incorporada	13
6	Construir una Web API REST	14
6.1	Parámetros, rutas y resultados	14
6.2	Agrupar endpoints relacionados	14

6.3	El estilo con controladores	15
6.4	Ejercicios — Web API	15
7	Acceso a datos con Entity Framework Core y PostgreSQL	16
7.1	Instalar los paquetes necesarios	16
7.2	El modelo y el DbContext	16
7.3	La cadena de conexión	17
7.4	Registrar el DbContext en el contenedor	17
7.5	Migraciones: del modelo a las tablas	18
7.6	Consultar y guardar con LINQ	18
7.7	Ejercicios — acceso a datos	19
8	Proyecto integrador: CRUD de estudiantes	19
8.1	Crear el proyecto y añadir dependencias	19
8.2	Modelo, DTO y contexto	20
8.3	Validación de la entrada	20
8.4	El Program.cs completo	21
8.5	Migrar y ejecutar	23
9	Documentar y probar la API con OpenAPI y Scalar	23
9.1	El cambio respecto a Swagger	23
9.2	Ya está configurado: cómo abrirlo	23
9.3	Solo en desarrollo	24
9.4	Probar con la línea de comandos	24
9.5	Ejercicios — documentación	24
10	Seguridad en la API	24
10.1	Autenticación con JWT	24
10.2	CORS: quién puede llamar desde el navegador	26
10.3	Buenas prácticas de seguridad	26
11	Panorama: MVC y Razor Pages	26
11.1	MVC	26
11.2	Razor Pages	27
12	Buenas prácticas y despliegue con Docker	27

12.1 Organización del código	27
12.2 Empaquetar con Docker	27
12.3 Ejercicios — seguridad y despliegue	28
13 Banco de ejercicios por nivel	28
13.1 Nivel básico	28
13.2 Nivel intermedio	29
13.3 Nivel avanzado	29
13.4 Proyectos propuestos	29
14 Soluciones seleccionadas	30
14.1 Endpoint de versión (básico)	30
14.2 Paginación del listado (intermedio)	30
14.3 Correo único con 409 (intermedio)	30
14.4 Login que emite un JWT (avanzado)	31
15 Glosario de siglas y términos	32
16 Recursos y referencias	34

1 Introducción

ASP.NET Core es la plataforma web de Microsoft: multiplataforma, de código abierto y con uno de los mejores rendimientos del mercado. Con ella, y usando el lenguaje C#, construirás desde un «Hola Mundo» de tres líneas hasta una API REST completa con base de datos, autenticación y documentación automática. Esta guía te lleva de cero a ese punto, explicando cada paso y proponiéndote ejercicios de todos los niveles para que practiques de verdad. A diferencia de otras guías del curso, aquí el IDE de referencia no es IntelliJ IDEA sino **Visual Studio Code**, porque es ligero, gratuito y el flujo de trabajo con la CLI de **dotnet** encaja perfectamente con él.

La estructura es progresiva: primero preparas el entorno, luego repasas lo justo de C# para la web, después construyes tu primer servicio, entiendes el *pipeline* de *middleware*, levantas una Web API REST, accedes a datos con Entity Framework Core y PostgreSQL, integras todo en un CRUD seguro y, por último, documentas y proteges tu API. Cada sección termina con ejercicios, y al final hay un banco de problemas por nivel con soluciones comentadas.

1.1 ¿Qué es ASP.NET Core y por qué dominarlo?

ASP.NET Core es un *framework* para construir aplicaciones y servicios web sobre la plataforma .NET. Microsoft lanzó el ASP.NET clásico en 2002 sobre el .NET Framework (solo Windows); en 2016 lo reescribió por completo como ASP.NET Core: modular, multiplataforma (Windows, Linux y macOS) y de código abierto. Desde .NET 5 (2020) la marca se unificó simplemente como «.NET», y la versión vigente en 2026 es **.NET 10**, una versión LTS (soporte de tres años) publicada en noviembre de 2025.

Dominarlo te aporta tres ventajas concretas. Primera, empleabilidad: .NET es uno de los ecosistemas más demandados en la industria, sobre todo en banca, salud y sector público. Segunda, rendimiento y robustez: C# es un lenguaje de tipado fuerte, y ASP.NET Core figura entre los *frameworks* web más rápidos en pruebas independientes. Tercera, transferencia de conceptos: los patrones que aprenderás aquí (peticiones, inyección de dependencias, acceso a datos, seguridad) son los mismos que reaparecen en Spring Boot (Java) o en Laravel (PHP), de modo que cada tecnología refuerza a las demás.

1.2 C# para quien ya sabe Java

Como en el curso ya dominas Java SE, la transición a C# es suave: ambos son lenguajes de tipado estático, orientados a objetos y con sintaxis de llaves muy parecida. Las diferencias más visibles son los nombres en mayúscula inicial para métodos y propiedades (**PascalCase**), las *propiedades* en lugar de *getters/setters* manuales, la palabra **var** para inferencia de tipos, los *records* para datos inmutables y el operador **=>** para expresiones y *lambdas*. A lo largo de la guía señalaremos estos puentes cuando aparezcan.

1.3 Cómo funciona ASP.NET Core en la web

Cuando un navegador o un cliente HTTP solicita una ruta de tu aplicación, la petición entra en un **pipeline** de componentes llamados *middleware*: cada uno la inspecciona, la modifica o la deja pasar al siguiente, hasta llegar al código que tú escribiste (un *endpoint*). Ese código produce una

respuesta, normalmente HTML o JSON, que recorre el *pipeline* en sentido inverso hasta el cliente. Entender este modelo de petición–respuesta y el orden del *pipeline* es la clave para no perderse después.

1.4 Cómo usar esta guía y los niveles de los ejercicios

Lee cada sección, reproduce los ejemplos en tu propio entorno y, sobre todo, resuelve los ejercicios: programar se aprende programando. Te recomendamos teclear el código en lugar de copiarlo, porque así detectas detalles que de otro modo pasan inadvertidos. Los ejercicios están etiquetados por nivel para que puedas dosificarlos:

- **[Básico]** afianzan la sintaxis y los conceptos esenciales; deberías resolverlos justo después de leer la sección.
- **[Intermedio]** combinan varios conceptos y se acercan a problemas reales (rutas, modelos, validación, datos).
- **[Avanzado]** exigen diseño, acceso a datos y seguridad; son los más parecidos a lo que harás en un proyecto.

2 Preparación del entorno de desarrollo

Antes de escribir una línea de código necesitas tres piezas: el **SDK de .NET 10**, el editor **Visual Studio Code** con sus extensiones de C#, y un servidor **PostgreSQL 18** para las prácticas con datos. A continuación se instalan y verifican una por una.

2.1 Versiones de referencia de esta guía

Para que el código sea reproducible, todo está probado contra las siguientes versiones. Usa siempre versiones con soporte activo por motivos de seguridad.

Componente	Versión de referencia (2026)
SDK de .NET	.NET 10 (LTS), soporte hasta noviembre de 2028
Lenguaje	C# 14
Framework web	ASP.NET Core 10
ORM	Entity Framework Core 10
Proveedor PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL 10.0.x
Base de datos	PostgreSQL 18
OpenAPI (documentación)	Microsoft.AspNetCore.OpenApi 10.0.x
UI de la API	Scalar.AspNetCore 2.x
Editor	Visual Studio Code (última estable)
Extensión principal	C# Dev Kit (incluye C# y <i>IntelliCode</i>)

Cambio importante en .NET 9/10: ya no viene Swagger por defecto

Si has visto tutoriales antiguos, sabrás que la plantilla `webapi` traía *Swashbuckle/Swagger*. Desde .NET 9 esto cambió: la plantilla ahora usa el generador de OpenAPI **integrado** (`AddOpenApi/MapOpenApi`) y *no* incluye ninguna interfaz visual. En esta guía añadiremos **Scalar** como UI moderna. Lo verás en detalle en la sección 9.

2.2 Paso 1: instalar el SDK de .NET 10

El SDK (*Software Development Kit*) incluye el compilador, las bibliotecas y la herramienta de línea de comandos `dotnet`, que es el centro de todo el flujo de trabajo. Descárgalo del sitio oficial de Microsoft e instálalo con las opciones por defecto.

Instalación del SDK

1. Entra en <https://dotnet.microsoft.com/download> y descarga el *SDK* de .NET 10 (no el *Runtime*: el SDK ya lo incluye).
2. Ejecuta el instalador con las opciones predeterminadas.
3. Cierra y vuelve a abrir cualquier terminal para que reconozca el comando `dotnet`.

Verifica que quedó bien instalado consultando la versión y la lista de SDK disponibles.

Verificar el SDK

```
1 dotnet --version
2 # 10.0.x
3
4 dotnet --list-sdks
5 # 10.0.x [C:\Program Files\dotnet\sdk]
```

2.3 Paso 2: instalar Visual Studio Code y las extensiones de C#

Visual Studio Code (VS Code) es un editor gratuito y ligero. Para programar en C# necesitas instalar una extensión: el **C# Dev Kit**, que a su vez instala la extensión base de C# y herramientas de productividad como el explorador de soluciones y el ejecutor de pruebas.

Instalar VS Code y C# Dev Kit

1. Descarga VS Code de <https://code.visualstudio.com> e instálalo.
2. Ábrelo y ve al panel de *Extensiones* (icono de bloques en la barra lateral, o `Ctrl+Shift+X`).
3. Busca **C# Dev Kit** (publicado por Microsoft) y pulsa *Install*. Se instalará junto con la extensión *C#* y *IntelliCode*.
4. Opcional pero recomendado: instala también **PostgreSQL** (de Microsoft o de Chris Kolkman) para inspeccionar la base de datos desde el propio editor.

Recarga tras instalar

Si VS Code ya estaba abierto cuando instalaste el SDK, ciérralo del todo y vuelve a abrirlo. La extensión de C# necesita encontrar el SDK en el PATH para activar el autocompletado y la depuración.

2.4 Paso 3: instalar PostgreSQL 18

Las prácticas con datos usan PostgreSQL, el mismo motor que el resto de las guías del curso. Si ya lo instalaste para la guía de Perl o PHP, puedes reutilizar esa instalación; solo necesitarás crear la base de datos del periodo.

Instalar y preparar PostgreSQL

1. Descarga PostgreSQL 18 de <https://www.postgresql.org/download> e instálalo. Anota la contraseña que asignes al usuario `postgres`.
2. Durante la instalación se incluye *pgAdmin*, una interfaz gráfica para administrar la base de datos.
3. Verifica que el servicio quedó arrancado y escuchando en el puerto 5432.

Crea la base de datos del periodo y la tabla `estudiantes` que usaremos en el proyecto integrador. Puedes hacerlo desde *pgAdmin* o desde la consola `psql`.

Esquema base en PostgreSQL

```

1  -- Base de datos del periodo academico
2  CREATE DATABASE appweb2026ppa;
3
4  -- Conectarse a ella y crear la tabla base
5  CREATE TABLE estudiantes (
6      id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
7      nombre     VARCHAR(120) NOT NULL,
8      correo     VARCHAR(160) NOT NULL UNIQUE,
9      carrera    VARCHAR(120) NOT NULL,
10     creado     TIMESTAMP NOT NULL DEFAULT NOW()
11 );

```

Identidad en vez de SERIAL

Desde PostgreSQL 10 se recomienda `GENERATED ALWAYS AS IDENTITY` en lugar del antiguo `SERIAL`: cumple el estándar SQL y es la opción que genera EF Core por defecto. Por eso la usamos aquí.

2.5 Paso 4: comprobar todo el entorno con un proyecto de prueba

Antes de avanzar, confirma que las tres piezas conversan entre sí creando, compilando y ejecutando un proyecto vacío. Esto valida el SDK y el editor de una sola vez.

Proyecto de prueba

```

1 # Crear una carpeta de trabajo y entrar en ella
2 mkdir prueba-entorno
3 cd prueba-entorno
4
5 # Crear un proyecto web mínimo y ejecutarlo
6 dotnet new web
7 dotnet run

```

La terminal mostrará una URL como `http://localhost:5xxx`. Ábrela en el navegador: deberías ver «Hello World!». Si llegaste hasta aquí, tu entorno está listo. Detén la aplicación con **Ctrl+C**.

3 Fundamentos de C# para la web

No necesitas ser un experto en C# para empezar, pero sí conocer media docena de construcciones que aparecerán en cada ejemplo. Como ya programas en Java, esta sección es un repaso rápido centrado en las diferencias.

3.1 Tipos, variables y propiedades

C# es de tipado estático: cada variable tiene un tipo. Puedes declararlo explícitamente o dejar que el compilador lo infiera con `var`. Las clases exponen su estado mediante *propiedades*, que son como *getters/setters* automáticos.

Tipos y propiedades

```

1 int edad = 21;           // tipo explícito
2 var nombre = "Ada";      // inferido como string
3 double promedio = 9.4;
4 bool activo = true;
5
6 // Una clase con propiedades (equivale a getter/setter de Java)
7 public class Estudiante
8 {
9     public int Id { get; set; }
10    public string Nombre { get; set; } = "";
11    public string Correo { get; set; } = "";
12 }

```

Records: datos inmutables en una línea

Para objetos que solo transportan datos (los DTO que verás luego), C# ofrece los **record**: declaran propiedades y comparación por valor automáticamente. Por ejemplo: `public record CrearEstudianteDto(string Nombre, string Correo, string Carrera);`

3.2 Métodos, lambdas y asincronía

Los métodos en C# usan **PascalCase**. El operador `=>` define tanto cuerpos de método de una sola expresión como *lambdas*. La asincronía se expresa con **async** y **await**, fundamentales en la web porque liberan el hilo mientras se espera la base de datos o la red.

Métodos y asincronía

```

1 // Metodo de una sola expresion
2 public int Doble(int n) => n * 2;
3
4 // Lambda asignada a una variable
5 Func<int, int> cuadrado = x => x * x;
6
7 // Metodo asincrono: devuelve Task y usa await
8 public async Task<string> LeerAsync()
9 {
10     await Task.Delay(100);    // simula espera de E/S
11     return "listo";
12 }
```

3.3 Colecciones y LINQ

Para listas usarás `List<T>`. LINQ (*Language Integrated Query*) permite consultar colecciones con una sintaxis declarativa parecida a SQL; más adelante, EF Core traducirá esas mismas consultas a SQL real contra PostgreSQL.

Colecciones y LINQ

```

1 var nombres = new List<string> { "Ada", "Alan", "Grace" };
2
3 // Filtrar y proyectar con LINQ
4 var conA = nombres
5     .Where(n => n.StartsWith("A"))
6     .OrderBy(n => n)
7     .ToList();           // ["Ada", "Alan"]
```

3.4 Ejercicios — fundamentos

Resuélvelos en un proyecto de consola (`dotnet new console`) para practicar el lenguaje antes de pasar a la web.

1. **[Básico]** Declara tres variables (un `int`, un `string` y un `bool`) e imprímelas con `Console.WriteLine`.
2. **[Básico]** Escribe un método `EsPar(int n)` que devuelva un `bool` usando `=>`.
3. **[Intermedio]** Crea un `record` `Producto(string Nombre, decimal Precio)` y una lista; muestra el más caro con LINQ.

4. **[Intermedio]** Dada una lista de números, devuelve solo los pares ordenados de mayor a menor con LINQ.

4 Primer proyecto web: Hola Mundo con Minimal API

Ya con el entorno listo, vamos a crear tu primera aplicación web «de verdad» y a entender cada archivo que genera la plantilla. Usaremos el modelo **Minimal API**, la forma más concisa de definir rutas en ASP.NET Core.

4.1 Crear y abrir el proyecto en VS Code

Todo proyecto .NET nace con la CLI. La crearemos desde la terminal y la abriremos en VS Code para trabajar con autocompletado y depuración.

Crear y abrir el proyecto

1. Abre una terminal en la carpeta donde guardas tus prácticas (por ejemplo D:\Repositorios\UTEQ).
2. Ejecuta los comandos de creación (abajo).
3. Abre la carpeta en VS Code con `code HolaWeb` o desde *File* → *Open Folder*.
4. La primera vez, VS Code preguntará si confías en la carpeta: acepta. La extensión de C# cargará el proyecto.

Crear el proyecto

```
1 dotnet new web -o HolaWeb
2 cd HolaWeb
3 code .
```

La plantilla `web` crea el proyecto Minimal API más pequeño posible. Su estructura es la siguiente.

Estructura creada por dotnet new web

```
1 HolaWeb/
2 |-- Program.cs           (punto de entrada; aquí programas)
3 |-- HolaWeb.csproj       (proyecto y dependencias NuGet)
4 |-- appsettings.json     (configuración de la app)
5 |-- appsettings.Development.json
6 |-- Properties/
7     |-- launchSettings.json (perfiles de ejecución y puertos)
```

4.2 Anatomía del archivo de proyecto (.csproj)

El archivo `.csproj` describe el proyecto en XML: qué versión de .NET usa y qué paquetes necesita. Es el equivalente al `pom.xml` de Maven que conoces de Java.

HolaWeb.csproj

```

1 <Project Sdk="Microsoft.NET.Sdk.Web">
2   <PropertyGroup>
3     <TargetFramework>net10.0</TargetFramework>
4     <Nullable>enable</Nullable>
5     <ImplicitUsings>enable</ImplicitUsings>
6   </PropertyGroup>
7 </Project>

```

Dos ajustes merecen atención. **Nullable** activa el análisis de referencias nulas, que te avisa en tiempo de compilación de posibles `NullReferenceException`. **ImplicitUsings** importa automáticamente los *namespaces* más comunes, de modo que no tengas que escribir `using System;` y similares en cada archivo.

4.3 El punto de entrada: Program.cs

Aquí vive tu código. La plantilla genera un **Program.cs** de pocas líneas que ya es una aplicación web funcional. Léelo con calma: cada línea importa.

Program.cs — generado por la plantilla

```

1 var builder = WebApplication.CreateBuilder(args);
2 var app = builder.Build();
3
4 app.MapGet("/", () => "Hello World!");
5
6 app.Run();

```

El **builder** configura la aplicación (servicios, configuración, *logging*); **Build** la construye; **MapGet** registra un *endpoint* que responde a peticiones GET en la ruta `/`; y **Run** arranca el servidor. La *lambda* `() => "Hello World!"` es el manejador: lo que devuelve es la respuesta.

4.4 Ejecutar y depurar desde VS Code

Tienes dos formas de ejecutar: por terminal con **dotnet run**, o con el depurador integrado (tecla F5), que además te permite poner puntos de interrupción.

Ejecutar con el depurador

1. Abre **Program.cs**.
2. Pulsa F5. Si VS Code pregunta el tipo de depuración, elige *C#*.
3. Se compilará, arrancará el servidor y se abrirá el navegador en la URL del perfil.
4. Para detener, pulsa el cuadrado rojo de la barra de depuración o **Shift+F5**.

Modifica el saludo y vuelve a ejecutar para comprobar el ciclo completo. Cambia la *lambda* para devolver HTML.

Devolver HTML en vez de texto

```
1 app.MapGet("/", () => Results.Content(  
2     "<!DOCTYPE html><html lang=\"es\"><head>" +  
3     "<meta charset=\"UTF-8\"><title>Hola</title></head>" +  
4     "<body><h1>Hola Mundo desde ASP.NET Core</h1></body></html>",  
5     "text/html"));
```

Puertos y launchSettings.json

Los puertos de desarrollo se definen en `Properties/launchSettings.json`. Allí verás perfiles `http` y `https` con sus URL. Puedes cambiar el puerto o pedir que el navegador se abra automáticamente en una ruta concreta (`launchUrl`).

4.5 Ejercicios — primer proyecto

1. [Básico] Añade un *endpoint* GET `/saludo` que devuelva tu nombre.
2. [Básico] Crea `/hora` que devuelva la fecha y hora actual con `DateTime.Now`.
3. [Intermedio] Crea `/suma/{a}/{b}` que reciba dos enteros en la ruta y devuelva su suma.
4. [Intermedio] Devuelve una página HTML con una lista de tres enlaces a tus otros *endpoints*.

5 El modelo de hosting y el pipeline de middleware

Toda petición que llega a ASP.NET Core atraviesa una cadena de componentes llamada *pipeline*. Cada componente es un *middleware*: una pieza que recibe la petición, decide qué hacer (registrar, autenticar, redirigir, responder) y, normalmente, la pasa al siguiente. Entender este orden evita la mayoría de los errores de configuración.

5.1 Qué es un middleware y por qué importa el orden

Imagina el *pipeline* como una serie de filtros en línea. La petición entra por el primero y avanza; la respuesta sale en sentido inverso. El orden en que registras los *middleware* en `Program.cs` es el orden en que se ejecutan, y es significativo: por ejemplo, la autenticación debe ir antes que la autorización, y los archivos estáticos suelen ir al principio.

Pipeline típico de una app web

```

1 var builder = WebApplication.CreateBuilder(args);
2 var app = builder.Build();
3
4 if (!app.Environment.IsDevelopment())
5 {
6     app.UseExceptionHandler("/error"); // manejo de errores en produccion
7     app.UseHsts(); // fuerza HTTPS en el navegador
8 }
9
10 app.UseHttpsRedirection(); // redirige http -> https
11 app.UseStaticFiles(); // sirve wwwroot/ (css, js, imagenes)
12 app.UseRouting(); // resuelve que endpoint atiende
13 app.UseAuthentication(); // identifica al usuario (quien es)
14 app.UseAuthorization(); // decide si puede (tiene permiso)
15
16 app.MapGet("/", () => "Hola"); // endpoints
17
18 app.Run();

```

Autenticación antes que autorización

`UseAuthentication` debe registrarse *antes* que `UseAuthorization`. Primero se establece *quién* es el usuario; solo después se evalúa *si* tiene permiso. Invertir el orden hace que la autorización no encuentre la identidad y rechace todo.

5.2 Inyección de dependencias incorporada

ASP.NET Core trae un contenedor de **inyección de dependencias** (DI) de serie. En lugar de crear tus servicios con `new` dentro de cada clase, los registras una vez en el `builder` y el *framework* te los entrega donde los necesites. Esto desacopla el código y facilita las pruebas. Los servicios se registran con tres ciclos de vida: `Singleton` (una instancia para toda la app), `Scoped` (una por petición) y `Transient` (una nueva cada vez).

Registrar y consumir un servicio

```

1 // 1) Declarar el contrato y su implementacion
2 public interface ISaludador { string Saludar(string nombre); }
3 public class Saludador : ISaludador
4 {
5     public string Saludar(string nombre) => $"Hola, {nombre}!";
6 }
7
8 // 2) Registrar en el contenedor (Program.cs)
9 builder.Services.AddScoped<ISaludador, Saludador>();
10
11 // 3) Consumir: el framework lo inyecta como parametro del endpoint
12 app.MapGet("/saludo/{nombre}", (string nombre, ISaludador svc) =>
13     svc.Saludar(nombre));

```

Fíjate en la cadena interpolada `$"Hola, {nombre}!"`: el prefijo `$` permite insertar variables entre

llaves dentro de la cadena, como en una plantilla.

6 Construir una Web API REST

Una **API REST** expone datos y operaciones a través de HTTP usando rutas y verbos (GET, POST, PUT, DELETE) y, normalmente, intercambia JSON. Es la base de cualquier *backend* moderno que luego consume un *frontend* (por ejemplo, en Angular). Veremos los dos estilos que ofrece ASP.NET Core: Minimal API y controladores.

6.1 Parámetros, rutas y resultados

Los *endpoints* reciben datos de tres formas: en la ruta (`/estudiantes/{id}`), en la cadena de consulta (`?buscar=ada`) o en el cuerpo (un JSON). ASP.NET Core enlaza cada uno automáticamente al parámetro correspondiente del manejador. Para responder se usa la clase **Results**, que produce los códigos de estado HTTP adecuados.

Enlace de parámetros y resultados

```
1 // Parametro de ruta
2 app.MapGet("/estudiantes/{id:int}", (int id) =>
3     Results.Ok(new { id, nombre = "Ada" }));
4
5 // Parametro de consulta (?buscar=...&pagina=...)
6 app.MapGet("/buscar", (string? buscar, int pagina = 1) =>
7     Results.Ok(new { buscar, pagina }));
8
9 // Cuerpo JSON enlazado a un record
10 app.MapPost("/estudiantes", (CrearEstudianteDto dto) =>
11     Results.Created($"/estudiantes/1", dto));
12
13 public record CrearEstudianteDto(string Nombre, string Correo, string Carrera);
```

La restricción `{id:int}` obliga a que el segmento sea un entero; si no lo es, ASP.NET Core responde 404 sin entrar a tu código. **Results.Ok** devuelve 200, **Results.Created** devuelve 201 con la cabecera **Location**, y **Results.NotFound** devuelve 404.

6.2 Agrupar endpoints relacionados

Cuando una entidad tiene varias operaciones, conviene agruparlas bajo un prefijo común con **MapGroup**. Esto reduce la repetición y permite aplicar configuración (como autorización) a todo el grupo de una vez.

Agrupar con MapGroup

```

1 var grupo = app.MapGroup("/v1/estudiantes");
2
3 grupo.MapGet("/", () => "lista");
4 grupo.MapGet("/{id:int}", (int id) => $"detalle {id}");
5 grupo.MapPost("/", (CrearEstudianteDto dto) => Results.Created("/", dto));

```

6.3 El estilo con controladores

Para APIs grandes, muchos equipos prefieren **controladores**: clases que agrupan acciones relacionadas y separan mejor las responsabilidades. Si vienes de Spring, los reconocerás de inmediato: equivalen a un `@RestController`. Se habilitan registrando `AddControllers` y mapeándolos con `MapControllers`.

Un controlador REST

```

1 using Microsoft.AspNetCore.Mvc;
2
3 [ApiController]
4 [Route("v1/[controller]")] // -> /v1/estudiantes
5 public class EstudiantesController : ControllerBase
6 {
7     [HttpGet]
8     public IActionResult Listar() => Ok(new[] { "Ada", "Alan" });
9
10    [HttpGet("/{id:int}")]
11    public IActionResult Detalle(int id) => Ok(new { id });
12
13    [HttpPost]
14    public IActionResult Crear([FromBody] CrearEstudianteDto dto)
15        => CreatedAtAction(nameof(Detalle), new { id = 1 }, dto);
16 }

```

¿Minimal API o controladores?

No hay una respuesta única. Las Minimal API son ideales para servicios pequeños y para aprender, porque todo está a la vista en `Program.cs`. Los controladores brillan en proyectos grandes, donde la organización por clases y los *filtros* aportan estructura. En esta guía usamos Minimal API para el proyecto integrador por claridad.

6.4 Ejercicios — Web API

1. **[Básico]** Crea un *endpoint* GET `/v1/ping` que devuelva `{ "estado": "ok" }` con `Results.Ok`.
2. **[Intermedio]** Implementa `/v1/eco` que reciba un JSON con un campo `mensaje` y lo devuelva con la hora.
3. **[Intermedio]** Agrupa con `MapGroup` cuatro *endpoints* de una entidad «curso» (lista, detalle,

crear, eliminar) que aún devuelvan datos simulados.

4. **[Avanzado]** Reescribe esos cuatro *endpoints* como un controlador con atributos `[HttpGet]`, `[HttpPost]` y `[HttpDelete]`.

7 Acceso a datos con Entity Framework Core y PostgreSQL

Las aplicaciones reales guardan y consultan datos en una base de datos. En .NET, la forma estándar de hacerlo es **Entity Framework Core** (EF Core), un ORM (*Object-Relational Mapper*): tú trabajas con objetos C# y EF Core traduce tus operaciones a SQL. Para PostgreSQL se usa el proveedor **Npgsql**. Es el equivalente, en el mundo .NET, a lo que JPA/Hibernate son en Java o Eloquent en Laravel.

7.1 Instalar los paquetes necesarios

EF Core no viene en la plantilla web; se añade como paquetes NuGet. Necesitas el proveedor de PostgreSQL y, para trabajar con migraciones desde la terminal, la herramienta `dotnet-ef`.

Añadir EF Core y Npgsql

```
1 # Proveedor de PostgreSQL para EF Core 10
2 dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
3
4 # Herramientas de diseño (migraciones)
5 dotnet add package Microsoft.EntityFrameworkCore.Design
6
7 # Instalar la CLI de EF Core de forma global (una sola vez)
8 dotnet tool install --global dotnet-ef
```

Comprobar la herramienta

Tras instalar `dotnet-ef`, verifica con `dotnet ef -version`. Si la terminal no lo encuentra, cierra y reábre la para que tome el `PATH` de las herramientas globales.

7.2 El modelo y el DbContext

Un **modelo** es una clase C# que representa una tabla; cada propiedad es una columna. El **DbContext** es la sesión con la base de datos: contiene un `DbSet<T>` por cada tabla y es a través de él que consultas y guardas.

Modelo Estudiante y ApplicationDbContext

```

1 using Microsoft.EntityFrameworkCore;
2
3 // Modelo: una fila de la tabla estudiantes
4 public class Estudiante
5 {
6     public int Id { get; set; }
7     public string Nombre { get; set; } = "";
8     public string Correo { get; set; } = "";
9     public string Carrera { get; set; } = "";
10    public DateTime Creado { get; set; }
11 }
12
13 // Contexto: la sesion con la base de datos
14 public class ApplicationDbContext : DbContext
15 {
16     public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
17         : base(options) { }
18
19     public DbSet<Estudiante> Estudiantes => Set<Estudiante>();
20 }

```

7.3 La cadena de conexión

La cadena de conexión describe cómo llegar a la base de datos. Se guarda en `appsettings.json` para no incrustarla en el código. Usamos la misma base `appweb2026ppa` del resto del curso.

appsettings.json

```

1 {
2     "ConnectionStrings": {
3         "Postgres": "Host=127.0.0.1;Port=5432;Database=appweb2026ppa;Username=postgres;Password=
4         P@$7Gr3$QL"
5     },
6     "Logging": {
7         "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" }
8     }
9 }

```

Nunca subas contraseñas al repositorio

En un proyecto real, la contraseña no va en `appsettings.json` versionado, sino en *User Secrets* (`dotnet user-secrets`) durante el desarrollo, o en variables de entorno en producción. Aquí la dejamos visible solo con fines didácticos y porque es la base local del curso.

7.4 Registrar el DbContext en el contenedor

El `DbContext` se registra como un servicio con ciclo de vida *Scoped* (uno por petición). Aquí le indicamos que use `Npgsql` con la cadena de conexión, y le decimos que apunte a PostgreSQL 18 para que aproveche sus características.

Registrar EF Core en Program.cs

```
1 builder.Services.AddDbContext<AppDbContext>(opciones =>
2     opciones.UseNpgsql(
3         builder.Configuration.GetConnectionString("Postgres"),
4         npg => npg.SetPostgresVersion(18, 0));
```

7.5 Migraciones: del modelo a las tablas

Una **migración** es un archivo que describe los cambios de esquema (crear tablas, añadir columnas) de forma versionada. EF Core las genera comparando tu modelo con el estado anterior, y luego las aplica a la base de datos. Es el flujo *code-first*: la fuente de verdad es tu código C#.

Crear y aplicar la primera migración

```
1 # Genera el código de la migración (carpeta Migrations/)
2 dotnet ef migrations add Inicial
3
4 # Aplica los cambios a la base de datos
5 dotnet ef database update
```

Si la tabla ya existe

Si creaste la tabla **estudiantes** a mano en la sección 2, EF Core intentará crearla de nuevo y fallará. Para estos ejercicios, deja que EF Core gestione el esquema: parte de una base vacía y aplica la migración, o usa una base distinta. Mantener una única fuente de verdad (el modelo) evita conflictos.

7.6 Consultar y guardar con LINQ

Con el `DbContext` inyectado, las operaciones CRUD se expresan con LINQ y métodos asíncronos. EF Core las traduce a SQL parametrizado, lo que te protege de la inyección SQL de forma automática.

CRUD con EF Core

```

1 // CREATE
2 var nuevo = new Estudiante { Nombre = "Ada", Correo = "ada@uteq.edu.ec",
3                               Carrera = "Software", Creado = DateTime.UtcNow };
4 db.Estudiantes.Add(nuevo);
5 await db.SaveChangesAsync();
6
7 // READ (todas, ordenadas)
8 var lista = await db.Estudiantes.OrderBy(e => e.Id).ToListAsync();
9
10 // READ (una por id)
11 var uno = await db.Estudiantes.FindAsync(1);
12
13 // UPDATE
14 uno!.Correo = "ada.lovelace@uteq.edu.ec";
15 await db.SaveChangesAsync();
16
17 // DELETE
18 db.Estudiantes.Remove(uno);
19 await db.SaveChangesAsync();

```

El sufijo `Async` y el `await` liberan el hilo mientras la base de datos responde, lo que permite atender más peticiones a la vez. Es la forma recomendada en la web.

7.7 Ejercicios — acceso a datos

Requieren PostgreSQL en marcha y la migración aplicada.

1. **[Intermedio]** Añade un *endpoint* GET `/v1/estudiantes` que devuelva todos los estudiantes ordenados por nombre.
2. **[Intermedio]** Inserta un estudiante de prueba al arrancar y comprueba en *pgAdmin* que se guardó.
3. **[Avanzado]** Implementa una búsqueda por carrera con `Where(e => e.Carrera == carrera)` recibida por *query string*.
4. **[Avanzado]** Crea un *endpoint* de conteo por carrera con `GroupBy` y proyección a un objeto anónimo.

8 Proyecto integrador: CRUD de estudiantes

Es hora de unir todo lo aprendido en una API REST completa que crea, lee, actualiza y elimina estudiantes en PostgreSQL, con validación y respuestas HTTP correctas. Partimos de un proyecto nuevo para que tengas el flujo de principio a fin.

8.1 Crear el proyecto y añadir dependencias

Creamos un proyecto Minimal API limpio y le añadimos EF Core, Npgsql y los paquetes de documentación que usaremos en la sección siguiente.

Preparar el proyecto

```

1 dotnet new web -o ApiEstudiantes
2 cd ApiEstudiantes
3 dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
4 dotnet add package Microsoft.EntityFrameworkCore.Design
5 dotnet add package Microsoft.AspNetCore.OpenApi
6 dotnet add package Scalar.AspNetCore
7 code .

```

8.2 Modelo, DTO y contexto

Separamos el **modelo** (lo que se guarda) de los **DTO** (*Data Transfer Objects*: lo que entra y sale por la API). Así no expones tu tabla directamente y controlas qué campos acepta el cliente. Crea un archivo `Datos.cs`.

Datos.cs — modelo DTO y contexto

```

1 using Microsoft.EntityFrameworkCore;
2
3 public class Estudiante
4 {
5     public int Id { get; set; }
6     public string Nombre { get; set; } = "";
7     public string Correo { get; set; } = "";
8     public string Carrera { get; set; } = "";
9     public DateTime Creado { get; set; }
10 }
11
12 // Lo que el cliente envia al crear o actualizar
13 public record EstudianteEntradaDto(string Nombre, string Correo, string Carrera);
14
15 // Lo que la API devuelve
16 public record EstudianteSalidaDto(int Id, string Nombre, string Correo,
17     string Carrera, DateTime Creado);
18
19 public class AppDbContext(DbContextOptions<AppDbContext> options)
20     : DbContext(options)
21 {
22     public DbSet<Estudiante> Estudiantes => Set<Estudiante>();
23 }

```

Observa el uso de **record** y del constructor primario en la declaración del contexto: son azúcar sintáctico de C# que reduce el código repetitivo respecto a Java, donde escribirías un constructor completo a mano.

8.3 Validación de la entrada

Antes de guardar, valida. Para un proyecto didáctico basta una función de validación que devuelva los errores encontrados; en proyectos grandes se usa la librería *FluentValidation*. Añade esta función auxiliar.

Validación sencilla

```

1 static Dictionary<string, string[]> Validar(EstudianteEntradaDto e)
2 {
3     var errores = new Dictionary<string, string[]>();
4     if (string.IsNullOrEmpty(e.Nombre))
5         errores["nombre"] = ["El nombre es obligatorio."];
6     if (string.IsNullOrEmpty(e.Correo) || !e.Correo.Contains('@'))
7         errores["correo"] = ["El correo no es valido."];
8     if (string.IsNullOrEmpty(e.Carrera))
9         errores["carrera"] = ["La carrera es obligatoria."];
10    return errores;
11 }

```

8.4 El Program.cs completo

Aquí está el corazón del proyecto: configuración de EF Core, OpenAPI y los cinco *endpoints* CRUD. Lo dividimos en dos partes para que sea legible. La primera parte configura los servicios y el *pipeline*.

Program.cs — parte 1 de 2: configuración

```

1 using Microsoft.EntityFrameworkCore;
2 using Scalar.AspNetCore;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 builder.Services.AddDbContext<AppDbContext>(o =>
7     o.UseNpgsql(builder.Configuration.GetConnectionString("Postgres"),
8         npg => npg.SetPostgresVersion(18, 0)));
9
10 builder.Services.AddOpenApi(); // genera /openapi/v1.json
11
12 var app = builder.Build();
13
14 if (app.Environment.IsDevelopment())
15 {
16     app.MapOpenApi(); // expone el documento OpenAPI
17     app.MapScalarApiReference(); // UI interactiva en /scalar/v1
18 }
19
20 app.UseHttpsRedirection();

```

La segunda parte define los *endpoints* agrupados bajo `/v1/estudiantes`. Cada uno recibe el `AppDbContext` por inyección de dependencias.

Program.cs — parte 2 de 2: endpoints CRUD

```

1  var grupo = app.MapGroup("/v1/estudiantes");
2
3  // READ: lista completa
4  grupo.MapGet("/", async (AppDbContext db) =>
5      await db.Estudiantes.OrderBy(e => e.Id)
6          .Select(e => new EstudianteSalidaDto(
7              e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado))
8          .ToListAsync());
9
10 // READ: uno por id
11 grupo.MapGet("/{id:int}", async (int id, AppDbContext db) =>
12     await db.Estudiantes.FindAsync(id) is Estudiante e
13     ? Results.Ok(new EstudianteSalidaDto(
14         e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado))
15     : Results.NotFound());
16
17 // CREATE
18 grupo.MapPost("/", async (EstudianteEntradaDto dto, AppDbContext db) =>
19 {
20     var errores = Validar(dto);
21     if (errores.Count > 0) return Results.ValidationProblem(errores);
22
23     var e = new Estudiante { Nombre = dto.Nombre, Correo = dto.Correo,
24                             Carrera = dto.Carrera, Creado = DateTime.UtcNow };
25     db.Estudiantes.Add(e);
26     await db.SaveChangesAsync();
27     return Results.Created($"v1/estudiantes/{e.Id}",
28         new EstudianteSalidaDto(e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado));
29 });
30
31 // UPDATE
32 grupo.MapPut("/{id:int}", async (int id, EstudianteEntradaDto dto,
33     AppDbContext db) =>
34 {
35     var errores = Validar(dto);
36     if (errores.Count > 0) return Results.ValidationProblem(errores);
37
38     var e = await db.Estudiantes.FindAsync(id);
39     if (e is null) return Results.NotFound();
40
41     e.Nombre = dto.Nombre; e.Correo = dto.Correo; e.Carrera = dto.Carrera;
42     await db.SaveChangesAsync();
43     return Results.NoContent();
44 });
45
46 // DELETE
47 grupo.MapDelete("/{id:int}", async (int id, AppDbContext db) =>
48 {
49     var e = await db.Estudiantes.FindAsync(id);
50     if (e is null) return Results.NotFound();
51     db.Estudiantes.Remove(e);
52     await db.SaveChangesAsync();
53     return Results.NoContent();
54 });
55
56 app.Run();

```

Cada *endpoint* devuelve el código HTTP correcto: 200 al leer, 201 al crear (con la cabecera *Location*), 204 al actualizar o borrar sin contenido, 404 cuando no existe y 400 con detalles cuando la validación falla. Esa precisión es lo que distingue una API profesional.

8.5 Migrar y ejecutar

Genera la migración, aplícala y arranca. Recuerda tener PostgreSQL en marcha y la cadena de conexión en `appsettings.json`.

Poner en marcha el CRUD

```
1 dotnet ef migrations add Inicial
2 dotnet ef database update
3 dotnet run
```

9 Documentar y probar la API con OpenAPI y Scalar

Una API sin documentación es difícil de usar. **OpenAPI** es un estándar que describe tu API en un documento JSON legible por máquinas; a partir de él, herramientas como **Scalar** generan una interfaz interactiva donde puedes ver y probar cada *endpoint* sin escribir código cliente.

9.1 El cambio respecto a Swagger

Como adelantamos, desde .NET 9 la plantilla dejó de incluir *Swashbuckle/Swagger*. Ahora el documento OpenAPI lo genera el paquete integrado `Microsoft.AspNetCore.OpenApi` mediante `AddOpenApi` y `MapOpenApi`, que publica el documento en `/openapi/v1.json`. Ese documento es solo datos: para una interfaz visual se añade una herramienta aparte. En esta guía usamos Scalar por su diseño moderno, su modo oscuro y su generador de ejemplos en varios lenguajes.

9.2 Ya está configurado: cómo abrirlo

En el `Program.cs` del proyecto integrador ya registraste `AddOpenApi`, `MapOpenApi` y `MapScalarApiReference`. Con la aplicación en marcha, abre las dos URL siguientes.

URLs de documentación

```
1 # Documento OpenAPI en JSON (lo consume cualquier herramienta)
2 https://localhost:5xxx/openapi/v1.json
3
4 # Interfaz interactiva de Scalar (probar endpoints desde el navegador)
5 https://localhost:5xxx/scalar/v1
```


Abrir Scalar al ejecutar

Para que el navegador abra Scalar automáticamente con F5, edita `Properties/launchSettings.json`: en el perfil `https`, pon `"launchBrowser": true` y `"launchUrl": "scalar/v1"`.

9.3 Solo en desarrollo

Fíjate en que la documentación se monta dentro de `if (app.Environment.IsDevelopment())`. Exponer la interfaz de tu API en producción revela su estructura interna y es una mala práctica de seguridad. Mantenla restringida al entorno de desarrollo, como hace la plantilla.

9.4 Probar con la línea de comandos

Además de Scalar, puedes probar la API con `curl` o con la extensión *REST Client* de VS Code. Aquí un ejemplo de creación con `curl`.

Probar con curl

```
1 curl -X POST https://localhost:5xxx/v1/estudiantes ^
2 -H "Content-Type: application/json" ^
3 -d '{"nombre": "Ada", "correo": "ada@uteq.edu.ec", "carrera": "Software"}'
```

9.5 Ejercicios — documentación

1. **[Básico]** Abre Scalar y ejecuta el GET de lista; observa la respuesta y el código de estado.
2. **[Intermedio]** Crea un estudiante desde Scalar y comprueba que devuelve 201 con la cabecera `Location`.
3. **[Intermedio]** Provoca un 400 enviando un correo sin @ y revisa el detalle de validación.
4. **[Avanzado]** Cambia el título y la versión del documento OpenAPI con un *document transformer* en `AddOpenApi`.

10 Seguridad en la API

Una API abierta a internet debe protegerse. Veremos tres pilares prácticos: autenticación con **JWT** (saber quién llama), **autorización** (decidir qué puede hacer) y **CORS** (controlar qué orígenes web pueden consumirla). Estos mecanismos son los mismos, conceptualmente, que en Spring Boot o Laravel.

10.1 Autenticación con JWT

Un JWT (*JSON Web Token*) es una credencial firmada que el cliente envía en la cabecera `Authorization` de cada petición. El servidor verifica la firma y, si es válida, confía en los datos del

token (por ejemplo, el usuario y su rol). Para usarlo, añade el paquete de autenticación JWT y configúralo.

Añadir y configurar JWT

```
1 dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

En `Program.cs`, registra la autenticación y la autorización, y añade los dos *middleware* en el orden correcto.

Configurar JWT en Program.cs

```
1 using Microsoft.AspNetCore.Authentication.JwtBearer;
2 using Microsoft.IdentityModel.Tokens;
3 using System.Text;
4
5 var clave = builder.Configuration["Jwt:Clave"]!; // secreto del servidor
6
7 builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
8     .AddJwtBearer(opt =>
9     {
10         opt.TokenValidationParameters = new TokenValidationParameters
11         {
12             ValidateIssuer = true,
13             ValidateAudience = true,
14             ValidateLifetime = true,
15             ValidateIssuerSigningKey = true,
16             ValidIssuer = "uteq-appweb",
17             ValidAudience = "uteq-estudiantes",
18             IssuerSigningKey = new SymmetricSecurityKey(
19                 Encoding.UTF8.GetBytes(clave))
20         };
21     });
22
23 builder.Services.AddAuthorization();
24
25 // ... despues de var app = builder.Build();
26 app.UseAuthentication(); // primero: quien es
27 app.UseAuthorization(); // despues: si puede
```

Para exigir un *token* válido en un *endpoint* o grupo, encadena `RequireAuthorization`. Las peticiones sin credencial válida recibirán 401 (no autenticado) o 403 (sin permiso).

Proteger endpoints

```
1 // Todo el grupo exige autenticacion
2 var grupo = app.MapGroup("/v1/estudiantes").RequireAuthorization();
3
4 // Un endpoint publico de salud, sin proteccion
5 app.MapGet("/salud", () => Results.Ok("ok"));
```

El secreto no va en el código

La clave de firma (`Jwt:Clave`) debe ser larga y aleatoria, y guardarse fuera del código: en *User Secrets* durante el desarrollo y en variables de entorno en producción. Si se filtra, cualquiera puede falsificar *tokens*.

10.2 CORS: quién puede llamar desde el navegador

CORS (*Cross-Origin Resource Sharing*) controla qué sitios web, desde otro origen, pueden consumir tu API desde el navegador. Si tu *frontend* en Angular corre en `http://localhost:4200` y tu API en otro puerto, necesitas autorizar ese origen explícitamente.

Configurar CORS

```

1 builder.Services.AddCors(o => o.AddPolicy("frontend", p =>
2     p.WithOrigins("http://localhost:4200")
3     .AllowAnyHeader()
4     .AllowAnyMethod()));
5
6 // despues de Build(), antes de los endpoints
7 app.UseCors("frontend");

```

No abras CORS a todo el mundo

Evita `AllowAnyOrigin` combinado con credenciales en producción: autoriza solo los orígenes que realmente consumen tu API. Una política demasiado permisiva expone tu *backend* a sitios maliciosos.

10.3 Buenas prácticas de seguridad

Más allá de JWT y CORS, recuerda estas reglas: usa siempre HTTPS (`UseHttpsRedirection`); valida toda entrada en el servidor; deja que EF Core parametrice las consultas (nunca concatenes SQL a mano); no expongas la documentación en producción; y devuelve mensajes de error genéricos al cliente, registrando el detalle solo en los *logs* del servidor.

11 Panorama: MVC y Razor Pages

Hasta aquí construimos una API que devuelve JSON. Pero ASP.NET Core también puede generar páginas HTML en el servidor con dos modelos clásicos. Conviene conocerlos aunque el proyecto del curso se centre en API + Angular.

11.1 MVC

En el patrón **MVC** (*Model-View-Controller*), un *controlador* recibe la petición, prepara un *modelo* de datos y elige una *vista* Razor (`.cshtml`) que lo renderiza a HTML. Es el modelo tradicional

para aplicaciones con muchas páginas y formularios. Se habilita con `AddControllersWithViews` y `MapControllerRoute`.

11.2 Razor Pages

Razor Pages simplifica MVC para escenarios centrados en páginas: cada página combina la plantilla (`.cshtml`) y su lógica (`.cshtml.cs`, llamado *PageModel*). Es muy productivo para formularios y paneles internos. Se habilita con `AddRazorPages` y `MapRazorPages`.

¿Cuál usar en el proyecto del curso?

Para el Proyecto Final de Curso (Spring Boot + Angular), la pieza en ASP.NET Core equivalente sería una **Web API** (lo que construiste aquí), consumida por un *frontend* en Angular. MVC y Razor Pages son útiles cuando el HTML se genera en el servidor, sin un *frontend* separado.

12 Buenas prácticas y despliegue con Docker

Cerramos con dos temas que marcan la diferencia entre un ejercicio y un proyecto profesional: la organización del código y el empaquetado para despliegue.

12.1 Organización del código

A medida que el proyecto crece, separa responsabilidades en carpetas: **Models/** para las entidades, **Data/** para el `DbContext`, **Dtos/** para los objetos de transferencia, **Services/** para la lógica de negocio y **Endpoints/** (o **Controllers/**) para las rutas. Extrae la lógica de los *endpoints* a servicios inyectables: deja los *endpoints* delgados y prueba la lógica por separado.

12.2 Empaquetar con Docker

Docker empaqueta tu aplicación y sus dependencias en una imagen que corre igual en cualquier máquina. Un `Dockerfile` de varias etapas compila en una imagen con el SDK y ejecuta en una imagen ligera con solo el *runtime*.

Dockerfile multi-etapa

```

1 # Etapa de compilacion
2 FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
3 WORKDIR /src
4 COPY . .
5 RUN dotnet publish -c Release -o /app
6
7 # Etapa de ejecucion (imagen ligera, solo runtime)
8 FROM mcr.microsoft.com/dotnet/aspnet:10.0
9 WORKDIR /app
10 COPY --from=build /app .
11 ENTRYPOINT ["dotnet", "ApiEstudiantes.dll"]

```

Publicar sin Docker

Si no usas contenedores, `dotnet publish -c Release` genera la aplicación lista para desplegar en una carpeta. El resultado se puede alojar tras un servidor inverso como Nginx, o en servicios en la nube que soportan .NET.

12.3 Ejercicios — seguridad y despliegue

1. **[Intermedio]** Protege el grupo `/v1/estudiantes` con `RequireAuthorization` y verifica que sin *token* responde 401.
2. **[Intermedio]** Añade una política CORS que autorice `http://localhost:4200` y pruébala desde un *fetch* en una página local.
3. **[Avanzado]** Extrae la lógica del CRUD a un `EstudianteService` inyectable y deja los *endpoints* delgados.
4. **[Avanzado]** Escribe el `Dockerfile`, construye la imagen y ejecuta el contenedor conectándolo a tu PostgreSQL.

13 Banco de ejercicios por nivel

Este banco reúne ejercicios adicionales para practicar de forma intensiva. Están agrupados por nivel; al final hay soluciones comentadas de algunos de ellos. Resuélvelos en tu entorno y verifícalos con `Scalar` o `curl`.

13.1 Nivel básico

Afianzan el lenguaje, las rutas y las respuestas. Deberías resolver cada uno en pocos minutos.

1. Crea un *endpoint* GET `/v1/version` que devuelva `{ "api": "estudiantes", "version": "1.0" }`.
2. Implementa `/v1/par/{n:int}` que devuelva si el número es par o impar.

3. Devuelve la fecha actual en formato ISO con `DateTime.UtcNow`.
4. Crea un `record` `Curso(int Id, string Nombre)` y un *endpoint* que devuelva una lista fija de tres cursos.
5. Añade `/v1/saludo` que reciba `?nombre=` y devuelva un saludo personalizado.
6. Crea `/v1/mayuscula/{texto}` que devuelva el texto en mayúsculas.

13.2 Nivel intermedio

Combinan modelos, validación, EF Core y documentación. Se parecen a tareas reales.

1. Añade un modelo `Carrera` con su `DbSet`, su migración y un CRUD básico.
2. Implementa paginación en el listado de estudiantes con `Skip` y `Take` y parámetros `pagina` y `tamano`.
3. Valida que el correo sea único antes de crear, devolviendo 409 si ya existe.
4. Crea una búsqueda por nombre con `Where(e => e.Nombre.Contains(texto))`.
5. Devuelve un `ProblemDetails` estándar (RFC 9457) en los errores de validación.
6. Añade ordenación configurable por *query string* (`?orden=nombre` o `?orden=carrera`).

13.3 Nivel avanzado

Exigen diseño, seguridad y arquitectura. Son los más cercanos a un proyecto profesional.

1. Implementa un *endpoint* de *login* que emita un JWT firmado y protege el CRUD con él.
2. Añade roles (`admin`, `consulta`) y restringe el borrado solo a `admin` con políticas de autorización.
3. Extrae toda la lógica a un `IEstudianteService` inyectable y escribe pruebas con `WebApplicationFactory`.
4. Implementa relaciones: un estudiante pertenece a una carrera (clave foránea) y carga relacionada con `Include`.
5. Añade *logging* estructurado de cada operación y un *middleware* global de manejo de errores.
6. Empaqueta la API con Docker y orquíestala junto a PostgreSQL con *docker compose*.

13.4 Proyectos propuestos

Para integrar todo en algo mayor, elige uno y desarróllalo por etapas, aplicando seguridad desde el inicio.

1. API de biblioteca: libros, préstamos y socios, con validación de disponibilidad en transacciones.
2. API de calificaciones: estudiantes, asignaturas y notas, con reportes por estudiante y por asignatura.

3. API de reservas de aulas con validación de solapamiento de horarios.
4. *Backend* para una agenda institucional consumido por un *frontend* en Angular.

14 Soluciones seleccionadas

Aquí tienes soluciones comentadas de algunos ejercicios para que contrastes tu enfoque. Casi siempre hay más de una solución válida; lo importante es que sea correcta, legible y segura.

14.1 Endpoint de versión (básico)

Devolvemos un objeto anónimo que ASP.NET Core serializa a JSON automáticamente.

Solución: endpoint de versión

```
1 app.MapGet("/v1/version", () =>
2     Results.Ok(new { api = "estudiantes", version = "1.0" }));
```

14.2 Paginación del listado (intermedio)

Usamos `Skip` y `Take` sobre la consulta. Calculamos el desplazamiento a partir del número de página y el tamaño.

Solución: paginación

```
1 grupo.MapGet("/", async (AppDbContext db, int pagina = 1, int tamano = 10) =>
2 {
3     if (pagina < 1) pagina = 1;
4     if (tamano < 1 || tamano > 100) tamano = 10;
5
6     var datos = await db.Estudiantes
7         .OrderBy(e => e.Id)
8         .Skip((pagina - 1) * tamano)
9         .Take(tamano)
10        .Select(e => new EstudianteSalidaDto(
11            e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado))
12        .ToListAsync();
13
14    var total = await db.Estudiantes.CountAsync();
15    return Results.Ok(new { pagina, tamano, total, datos });
16 });
```

14.3 Correo único con 409 (intermedio)

Antes de crear, comprobamos si el correo ya existe con `AnyAsync`. Si existe, devolvemos 409 *Conflict*.

Solución: validar correo único

```
1 grupo.MapPost("/", async (EstudianteEntradaDto dto, AppDbContext db) =>
2 {
3     var errores = Validar(dto);
4     if (errores.Count > 0) return Results.ValidationProblem(errores);
5
6     bool existe = await db.Estudiantes.AnyAsync(e => e.Correo == dto.Correo);
7     if (existe)
8         return Results.Conflict(new { mensaje = "El correo ya esta registrado." });
9
10    var e = new Estudiante { Nombre = dto.Nombre, Correo = dto.Correo,
11                           Carrera = dto.Carrera, Creado = DateTime.UtcNow };
12    db.Estudiantes.Add(e);
13    await db.SaveChangesAsync();
14    return Results.Created($" /v1/estudiantes/{e.Id}",
15        new EstudianteSalidaDto(e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado));
16 });
```

14.4 Login que emite un JWT (avanzado)

Tras validar credenciales (aquí simplificadas), construimos un *token* firmado con la clave del servidor y unos *claims* básicos.

Solución: emitir un JWT

```

1 using System.IdentityModel.Tokens.Jwt;
2 using System.Security.Claims;
3 using Microsoft.IdentityModel.Tokens;
4 using System.Text;
5
6 app.MapPost("/v1/login", (LoginDto dto, IConfiguration cfg) =>
7 {
8     // En un proyecto real: verificar usuario y hash de la contraseña en BD
9     if (dto.Usuario != "admin" || dto.Clave != "secreto")
10         return Results.Unauthorized();
11
12     var clave = new SymmetricSecurityKey(
13         Encoding.UTF8.GetBytes(cfg["Jwt:Clave"]!));
14     var credenciales = new SigningCredentials(
15         clave, SecurityAlgorithms.HmacSha256);
16
17     var claims = new[]
18     {
19         new Claim(ClaimTypes.Name, dto.Usuario),
20         new Claim(ClaimTypes.Role, "admin")
21     };
22
23     var token = new JwtSecurityToken(
24         issuer: "uteq-appweb",
25         audience: "uteq-estudiantes",
26         claims: claims,
27         expires: DateTime.UtcNow.AddHours(2),
28         signingCredentials: credenciales);
29
30     var jwt = new JwtSecurityTokenHandler().WriteToken(token);
31     return Results.Ok(new { token = jwt });
32 });
33
34 public record LoginDto(string Usuario, string Clave);

```

Copia el *token* devuelto y, en Scalar, pégalo en la opción de autenticación *Bearer* para llamar a los *endpoints* protegidos. Es la base de un *backend* seguro que luego consume un *frontend*.

15 Glosario de siglas y términos

Este glosario reúne las siglas y términos que aparecen en la guía, para consulta rápida durante el estudio.

Término	Significado
.NET	Plataforma de desarrollo de Microsoft (runtime, bibliotecas y herramientas).
ASP.NET Core	Framework web de .NET, multiplataforma y de código abierto.
C#	Lenguaje principal de .NET, de tipado estático y orientado a objetos.

Término	Significado
SDK	Software Development Kit: compilador, bibliotecas y CLI <code>dotnet</code> .
CLI	Command-Line Interface (la herramienta <code>dotnet</code>).
Minimal API	Estilo conciso para definir rutas directamente en <code>Program.cs</code> .
Middleware	Componente del <i>pipeline</i> que procesa cada petición y respuesta.
DI	Inyección de dependencias (<i>Dependency Injection</i>).
REST	Estilo de API basado en recursos, rutas y verbos HTTP.
DTO	Data Transfer Object: objeto para entrada/salida de la API.
EF Core	Entity Framework Core: el ORM de .NET.
ORM	Object-Relational Mapper: mapea objetos a tablas.
Npgsql	Proveedor de PostgreSQL para ADO.NET y EF Core.
DbContext	Sesión con la base de datos en EF Core.
Migración	Archivo versionado que describe cambios de esquema en EF Core.
LINQ	Language Integrated Query: consultas declarativas en C#.
OpenAPI	Estándar para describir APIs en un documento legible por máquinas.
Scalar	Interfaz interactiva moderna para documentos OpenAPI.
Swagger	Conjunto de herramientas OpenAPI; ya no viene por defecto desde .NET 9.
JWT	JSON Web Token: credencial firmada para autenticación.
CORS	Cross-Origin Resource Sharing: control de orígenes web permitidos.
MVC	Model-View-Controller: patrón para generar HTML en el servidor.
Razor Pages	Modelo de páginas (<code>.cshtml</code> + <i>PageModel</i>) de ASP.NET Core.
Docker	Empaqueta la aplicación y sus dependencias en una imagen portable.

16 Recursos y referencias

Para seguir aprendiendo y consultar dudas, estos son los recursos oficiales más fiables del ecosistema .NET. Todas las direcciones corresponden a documentación o repositorios verificados.

- Documentación oficial de .NET: **learn.microsoft.com/dotnet**
- Documentación de ASP.NET Core 10: **learn.microsoft.com/aspnet/core**
- Descarga del SDK de .NET: **dotnet.microsoft.com/download**
- Entity Framework Core: **learn.microsoft.com/ef/core**
- Proveedor Npgsql para EF Core: **npgsql.org/efcore**
- OpenAPI en ASP.NET Core: **learn.microsoft.com/aspnet/core/fundamentals/openapi**
- Scalar para .NET: **github.com/scalar/scalar**
- PostgreSQL: **postgresql.org/docs**
- Visual Studio Code para C#: **code.visualstudio.com/docs/languages/csharp**
- Seguridad web (OWASP Top 10): **owasp.org/Top10**

Con esta guía y la práctica constante de los ejercicios, tendrás una base sólida para desarrollar APIs web con ASP.NET Core y para integrarlas, más adelante, con un *frontend* en Angular en el Proyecto Final de Curso.